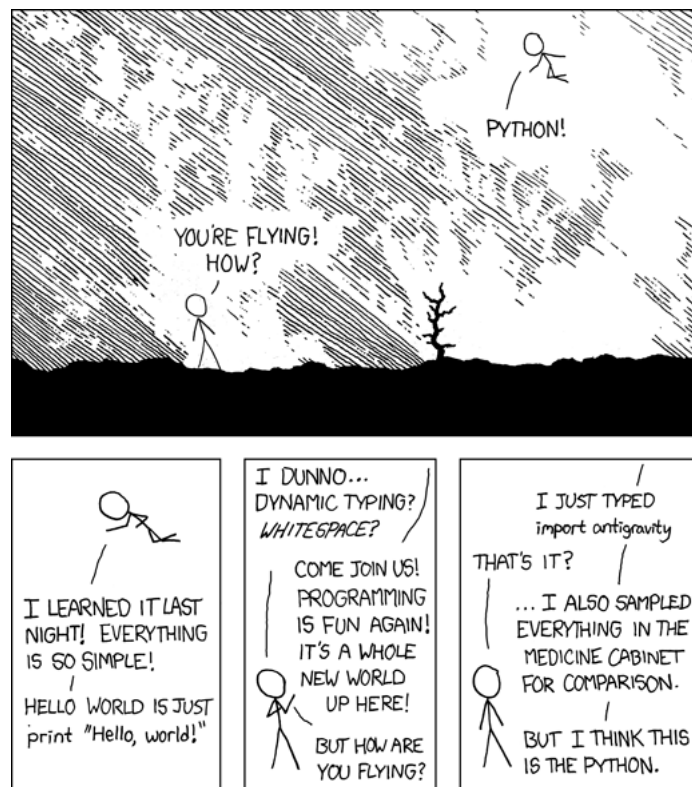


MPI* Maths
Préparation Centrale
Oral Maths-Python



Olivier Caffier - Pierre Bodet



Mis à jour le 10 juin 2025

0 Préambule

Ce polycopié contient à peu près tous les exercices proposés par Monsieur Pascal Vallaëys, professeur de Mathématiques en MPI* au lycée Faidherbe, dans le cadre de la préparation aux oraux Maths-Informatique du concours Centrale-Supélec. Ce travail a été réalisé par des élèves dans un temps limité donc n'est évidemment pas exempté d'erreurs, nous vous invitons à nous le signaler le cas échéant.

Les modules utilisés dans l'entête de nos fichiers sont les suivants :

```
1 # Importation des modules - Olivier
2
3 import numpy as np
4 import numpy.linalg as alg
5 import numpy.random as rd
6 from numpy.polynomial import Polynomial
7
8
9 import math
10 import matplotlib.pyplot as plt
11 from mpl_toolkits.mplot3d import Axes3D
12
13
14 import scipy.optimize as resol
15 import scipy.integrate as integr
```

```
1 # Importation des modules - Pierre
```

- 1 Algèbre linéaire**
- 2 Suites et séries numériques**

3 Fonctions d'une variable réelle

Exercice 8 Écrire une fonction qui à $P \in \mathbb{C}[X]$ renvoie $t \mapsto P(e^{it})$ et tester avec $P = X^2 - X + \frac{1}{2}$.

Corrigé 8 On a la fonction suivante :

```
1 def f_polynome (P) :
2     """
3     Prend un polynome P et renvoie f_P une fonction qui prend t comme variable
4     """
5     return lambda t: P(np.exp(1j * t))
6
7
8 # Définir le polynome P(X) = X^2 - X + 1/2
9 P = Polynomial([1, -1, 0.5])
10
11 # Obtenir la fonction t -> P(e^{it})
12 f = f_polynome(P)
13
14 # Tester pour quelques valeurs de t
15 t_vals = np.linspace(0, 2 * np.pi, 300)
16 f_vals = f(t_vals)
17
18 # Affichage des parties réelle et imaginaire
19 plt.plot(t_vals, f_vals.real, label="Re(P(e^{it}))")
20 plt.plot(t_vals, f_vals.imag, label="Im(P(e^{it}))")
21 plt.title("P(e^{it}) avec P(X) = X^2 - X + 1/2")
22 plt.xlabel("t")
23 plt.ylabel("Valeur complexe")
24 plt.legend()
25 plt.grid(True)
26 plt.show()
```

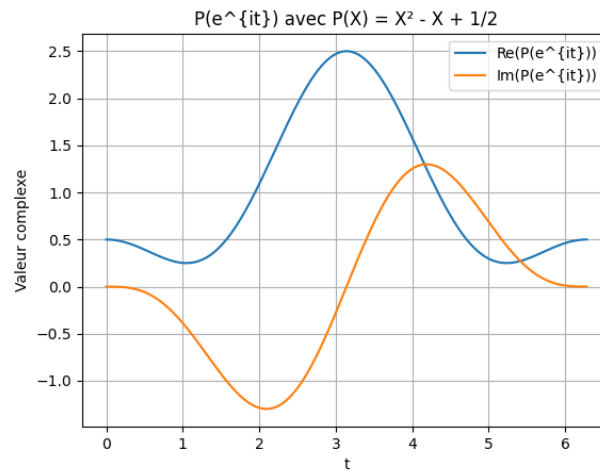


FIGURE 1 – Tracé de $t \mapsto \Re(P(e^{it}))$ et $t \mapsto \Im(P(e^{it}))$

Exercice 9 Soient $H_0 = 1$ et $H_1 = X$ deux polynômes. On définit la famille $(H_k)_{k \in \mathbb{N}}$ par :

$$\forall k \in \mathbb{N}, H_{k+2} = XH_{k+1} - \frac{k+1}{2}H_k$$

On représente un polynôme $a_0 + \dots + a_n X^n$ avec $a_n \neq 0$ par la liste $L = [a_0, \dots, a_n]$ sur Python. On s'interdit le module `Numpy`. `Polynomial`.

1. Coder :

- `Enleve_Zeros(P)` qui enlève de P les 0 inutiles;
- `Mult(a,P)` qui renvoie la liste qui représente aP ;
- `MultX(P)` qui renvoie la liste qui représente XP ;
- `Somme(P,Q)` qui renvoie la liste qui représente $P + Q$;
- `Derive(P)` qui renvoie la liste qui représente P' .

2. Déterminer les dix premiers polynômes de la famille définie. Donner le coefficient dominant et le degré de ces polynômes.

Tracer les H_k pour k allant de 1 à 5 et déterminer une condition que les racines des H_k vérifient.

Corrigé 9

1. On a les fonctions suivantes :

```

1 def Enleve_Zeros(P) :
2     return [b for b in P if b != 0]
3
4 def Mult(a,P) :
5     return [a * b for b in P]
6
7 def MultX(P) :
8     res = P.copy()
9     return [0] + res
10
11 def Somme(P, Q):
12     '''
13     Somme les polynomes P et Q
14     '''
15     n_p, n_q = len(P), len(Q)
16     n = max(n_p, n_q)
17     res = []
18     for i in range(n):
19         coeff_P = P[i] if i < n_p else 0 # Si on depasse la taille de P, il n'y a plus de coeff
20         coeff_Q = Q[i] if i < n_q else 0 #
21         res.append(coeff_P + coeff_Q)
22
23     return res
24
25 def Derive(P) :
26     return [i * P[i] for i in range(1,len(P))] # On evite Enleve_Zeros car sinon on obtiendrait pas P'
```

2. On utilisera une fonction `Genere(n)` qui renverra la liste des n polynômes recherchés (ici $n = 10$).

```

1 def Genere(n) :
2     #Genere les H_k
3     res = [[1], [0,1]]
4     for i in range(2,n) :
5         new_p = Somme(MultX(res[i-1]), Mult(- (i-1)/2, res[i-2]))
6         res.append(new_p)
7     return res
8
9 def Coef_dom(P) :
10     # Donne le coeff dominant d'un polynome
11     return P[len(P)-1]
12
13 def deg(P) :
14     # Donne le degre d'un polynome
15     return len(P) -1
16
```

```

17 # Exemples de H_1 jusqu' H_10
18 prem_10 = Genere(10)
19 cd_H10, deg_H10 = [Coef_dom(P) for P in prem_10], [deg(P) for P in prem_10]
20
21 def evaluate(P):
22     """
23     Retourne une fonction qui evalue le polynome represente par la liste P
24     (coefficients de degre croissant) sur un scalaire ou un tableau t.
25     """
26     def f(t):
27         t = np.asarray(t)
28         res = np.zeros_like(t, dtype=np.float64)
29         for i in range(len(P)-1, -1, -1): # Horner: du plus haut degre au plus bas
30             res = res * t + P[i]
31         return res
32     return f
33
34 # Trace des fonctions
35 h0 = evaluate(prem_10[0])
36 h1 = evaluate(prem_10[1])
37 h2 = evaluate(prem_10[2])
38 h3 = evaluate(prem_10[3])
39 h4 = evaluate(prem_10[4])
40
41 t_vals = np.linspace(-1,1,300)
42 h0_vals = h0(t_vals)
43 h1_vals = h1(t_vals)
44 h2_vals = h2(t_vals)
45 h3_vals = h3(t_vals)
46 h4_vals = h4(t_vals)
47
48 plt.clf()
49 plt.plot(t_vals,h0_vals, label = r"$H_0(t)$")
50 plt.plot(t_vals,h1_vals, label = r"$H_1(t)$")
51 plt.plot(t_vals,h2_vals, label = r"$H_2(t)$")
52 plt.plot(t_vals,h3_vals, label = r"$H_3(t)$")
53 plt.plot(t_vals,h4_vals, label = r"$H_4(t)$")
54
55 plt.title(r"Trace des premiers $H_k(t)$")
56 plt.xlabel("t")
57 plt.legend()
58 plt.grid(True)
59 plt.show()

```

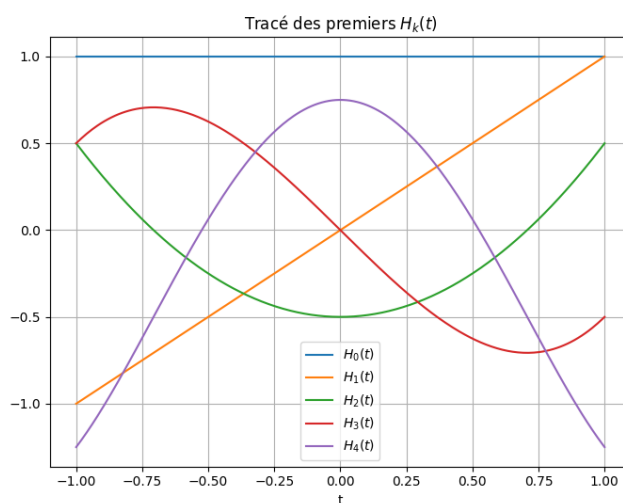


FIGURE 2 – Tracé des 5 premiers H_k

Exercice 10 On considère la suite de polynômes suivante :

$$\begin{cases} T_0 = 1 \\ T_1 = X \\ \forall n \in \mathbb{N}, T_{n+2} = 2XT_{n+1} - T_n \end{cases}$$

1. Écrire en Python une fonction `fonction(n)` telle que pour $n \in \mathbb{N}$, renvoie le polynôme T_n . On utilisera la classe `Polynomial` du module `numpy.polynomial`.
2. Tracer T_1, T_2, T_3, T_4, T_5 . Combien de racines ont-ils ?
3. Vérifier (toujours avec Python) sur quelques valeurs de $n, m \in \mathbb{N}$ que $T_n \circ T_m = T_m \circ T_n$

Corrigé 10

1. On a la fonction suivante :

```

1 def fonction(n) :
2     if n == 0 :
3         return Polynomial([1])
4     elif n == 1 :
5         return Polynomial([0,1])
6     else :
7         t_minus_2 = Polynomial([1])
8         t_minus_1 = Polynomial([0,1])
9         ind = 2
10        while ind <= n :
11            t_tmp = 2 * Polynomial([0,1]) * t_minus_1 - t_minus_2
12            ind += 1
13            t_minus_2 = t_minus_1
14            t_minus_1 = t_tmp
15        return t_minus_1

```

2. On a l'affichage suivant :

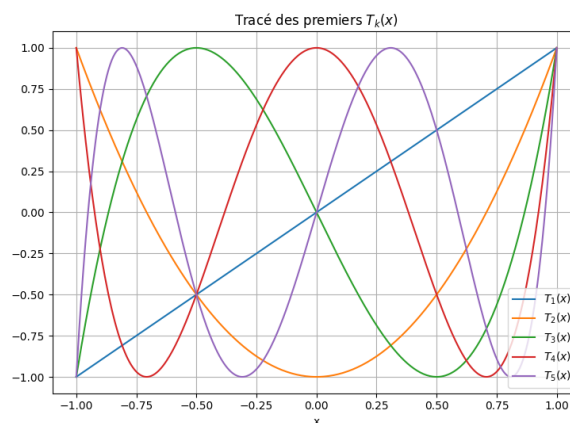


FIGURE 3 – Tracé des polynômes de Tcheb

```

1 x_vals = np.linspace(-1,1,300)
2
3 t1 = fonction(1)
4 t1_vals = t1(x_vals)
5 t2 = fonction(2)
6 t2_vals = t2(x_vals)
7 t3 = fonction(3)
8 t3_vals = t3(x_vals)
9 t4 = fonction(4)
10 t4_vals = t4(x_vals)
11 t5 = fonction(5)
12 t5_vals = t5(x_vals)
13

```

```

14 plt.clf()
15 plt.plot(x_vals,t1_vals, label = r"$T_1(x)$")
16 plt.plot(x_vals,t2_vals, label = r"$T_2(x)$")
17 plt.plot(x_vals,t3_vals, label = r"$T_3(x)$")
18 plt.plot(x_vals,t4_vals, label = r"$T_4(x)$")
19 plt.plot(x_vals,t5_vals, label = r"$T_5(x)$")
20
21 plt.title(r"Trace des premiers $T_k(x)$")
22 plt.xlabel("x")
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Pour les racines : soit on compte graphiquement, soit on utilise la fonction `.roots()` qui donne un array avec les racines du polynômes. On retrouve bien la propriété classique des polynômes de Tcheb :

$$\forall n \in \mathbb{N}, \# \text{Rac}(T_n) = n$$

3. On va juste montrer la coïncidence sur $[-1, 1]$ et on pourra conclure (intervalle non réduit à un point) :

```

1 t5_t2_vals = t5(t2(x_vals))
2 t2_t5_vals = t2(t5(x_vals))
3 plt.clf()
4 plt.plot(x_vals,t5_t2_vals, label = r"$T_5 \circ T_2 (x)$")
5 plt.plot(x_vals,t2_t5_vals, label = r"$T_2 \circ T_5 (x)$")
6
7
8 plt.title(r"Vérification que $T_5 \circ T_2 = T_2 \circ T_5$")
9 plt.xlabel("x")
10 plt.legend()
11 plt.grid(True)
12 plt.show()

```

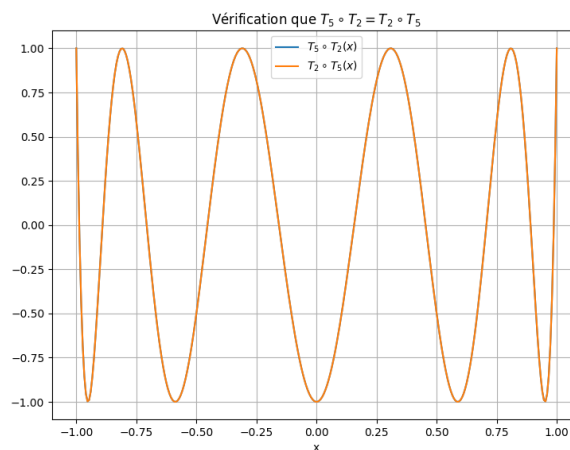


FIGURE 4 – Commutativité de T_2 et T_5

D'où le résultat voulu (ABSL pour les autres).

4 Notion de norme

Exercice 14 La norme euclidienne sur $\mathbb{R}^2$ est notée $\|\cdot\|$. On pose pour $x \in \mathbb{R}_+$:

$$\begin{cases} C(x) = \{u \in \mathbb{Z}^2 \mid \|u\| \leq x\} \\ C^+(x) = C(x) \cap \mathbb{N}^2 \end{cases}$$

et on note :

$$\begin{cases} N(x) = \#C(x) \\ N^+(x) = \#C^+(x) \end{cases}$$

1. Écrire un programme Python qui calcule $N(x)$ et $N^+(x)$.
2. Vérifier numériquement que $4N^+(x) - N(x) - 4\lfloor x \rfloor$ est constant.
3. Tracer le graphe de $x \mapsto \frac{N(x)}{x^2}$

Corrigé 14

1. On a le code suivant :

```
1 def norm_eucl(z1,z2):
2     return np.sqrt(z1**2 + z2**2)
3
4 def tuple_norme_C(x) :
5     res = []
6     n = int(np.ceil(x)) # Partie entiere superieure
7     for i in range(-n ,n + 1) :
8         for j in range(-n , n+1) :
9             if norm_eucl(i,j) <= x :
10                 res.append((i,j))
11     return res
12
13 def norme_N(x) :
14     return len(tuple_norme_C(x))
15
16 def norme_N_plus(x) :
17     tmp = tuple_norme_C(x)
18     res_to_count = [(i,j) for (i,j) in tmp if (i>=0) & (j >= 0)]
19     return len(res_to_count)
```

2. On vérifie cela sur l'intervalle $[0,20]$:

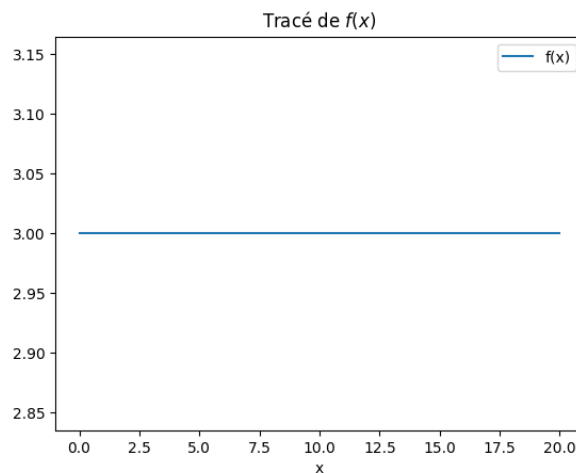


FIGURE 5 – Tracé de $f : x \mapsto 4N^+(x) - N(x) - 4\lfloor x \rfloor$

Attention! Ici la fonction n'est pas vectorisée, donc on ne peut pas directement appliquer f à un `np.array`. Pour cela, on utilisera `f_vect = np.vectorize(f)`.

3. Il ne reste plus qu'à définir et tracer cette fonction, cette fois-ci sur $[1,20]$ pour ne pas diviser par 0 :

```

1 def g(x) :
2     return norme_N(x)/(x**2)
3
4 t_vals = np.linspace(1,50,300)
5 #f_vect = np.vectorize(f)
6 #f_vals = f_vect(t_vals)
7
8 g_vect = np.vectorize(g)
9 g_vals = g_vect(t_vals)
10
11 plt.clf()
12 #plt.plot(t_vals,f_vals,label = r"f(x)")
13 plt.plot(t_vals,g_vals,label = r"g(x)")
14 plt.title(r"Trace de $g(x)$")
15 plt.grid(True)
16 plt.xlabel("x")
17 plt.legend()
18 plt.show()

```

et on obtient le graphique suivant :

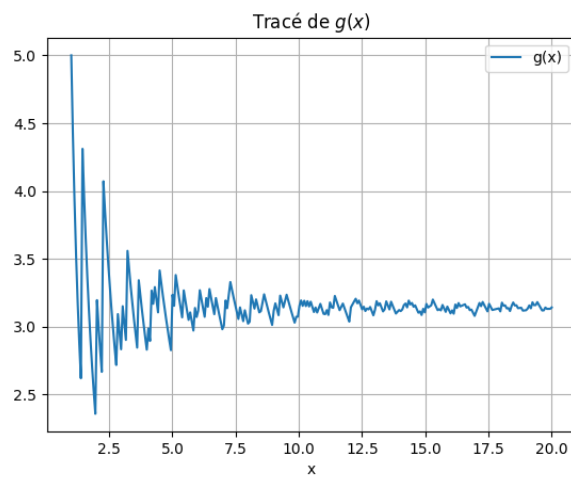


FIGURE 6 – Tracé de $g : x \mapsto \frac{N(x)}{x^2}$

5 Espaces probabilisés

Exercice 25 Deux joueurs A et B s'affrontent lors d'un match de badminton selon les règles suivantes :

- A commence à servir
- le joueur qui gagne le point sert pour le point suivant
- le joueur qui sert a probabilité $2/3$ de gagner le point et $1/3$ de le perdre

Pour $n \in \mathbb{N}^*$, on note A_n (resp. B_n) l'événement « A (resp. B) gagne le n-ème point » ainsi que $a_n = P(A_n)$ et $b_n = P(B_n)$.

5. Grâce à python, représenter graphiquement les dix premiers termes des suites (a_n) et (b_n) .

Corrigé 25

On a prouvé dans la question 3. que $\begin{pmatrix} a_{n+1} \\ b_{n+1} \end{pmatrix} = M \begin{pmatrix} a_n \\ b_n \end{pmatrix}$ avec $M = \begin{pmatrix} 2/3 & 1/3 \\ 1/3 & 2/3 \end{pmatrix}$

```
1 # Calcul des valeurs
2 M = np.array([[2/3, 1/3], [1/3, 2/3]])
3 # a1 sera present dans a[0]
4 a = np.zeros(10)
5 b = np.zeros(10)
6 a[0] = 2/3
7 b[0] = 1/3
8 for i in range(9): # on veut generer 9 valeurs car a1 et b1 sont deja connus
9     etat_actuel = np.array([a[i], b[i]])
10    etat_suivant = np.dot(M, etat_actuel)
11    a[i+1] = etat_suivant[0]
12    b[i+1] = etat_suivant[1]
13
14 # Affichage des valeurs
15 x = np.array([i for i in range(1,11)])
16 plt.plot(x,a,color='r',linestyle=':',marker='o',label="(an)")
17 plt.plot(x,b,color='b',linestyle=':',marker='o',label="(bn)")
18 plt.title("10 premiers termes des suites (an) et (bn)")
19 plt.legend()
20 plt.show()
```

et on obtient le graphique suivant :

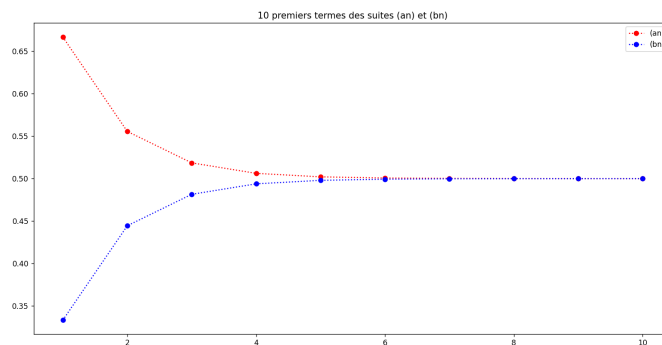


FIGURE 7 – 10 premières valeurs de (a_n) et (b_n)

Exercice 26 On note S_n^k le nombre de surjections existant entre un ensemble à n éléments et un ensemble à k éléments.

5. Implémenter un programme python permettant de calculer les S_n^k .

Corrigé 26

On a prouvé dans la question 4. que $S_n^k = k \left(S_{n-1}^k + S_{n-1}^{k-1} \right)$. On en déduit ainsi le code suivant :

```
1 def s(n,k):
2     res = np.zeros((n+1,k+1),dtype=int) # Les elements d'indice 0 ne seront pas utilises
3     # Conditions initiales d'apres la question 1
4     for i in range(n+1):
5         res[i,1] = 1
6     for i in range(k+1):
7         res[i,i] = np.math.factorial(i)
8     # Relation de recurrence
9     for i in range(n+1):
10        for j in range(min(k+1,i)): # On s'arrete a la diagonale car au-dela, il n'y a pas de surjection qui
            convient
11            res[i,j] = j * (res[i-1,j] + res[i-1,j-1])
12    return res[n,k]
```

6 Espaces euclidiens

Exercice 27 On considère $E = \mathbb{R}^3$ muni de la norme euclidienne. Soit $q : E \rightarrow \mathbb{R}$ telle que : $\forall u = (x, y, z) \in E, q(u) = x^2 + y^2 - z^2$. Soit $C = \{u \in E \mid q(u) = 0\}$ et $D = \{u \in \mathbb{C} \mid \|u\| = 1\}$.

1) Tracer la surface C avec Python. De quel type de surface s'agit-il?

4) Représenter D à l'aide de Python.

Corrigé 27 1)

```
1 ax = Axes3D(plt.figure())
2
3 def f(r, theta):
4     # En cartésienne, f = x^2 + y^2 donc vaut r en cylindrique
5     return r
6
7 r = np.arange(0, 1, 0.02)
8 theta = np.arange(0, 2*np.pi, 0.02)
9 R, Theta = np.meshgrid(r, theta) # R est une matrice ou chaque ligne vaut r et
10                                     # Theta est une matrice ou chaque colonne vaut theta
11 X = R * np.cos(Theta)
12 Y = R * np.sin(Theta)
13 Zpos = f(R, Theta)
14 Zneg = -f(R, Theta)
15 ax.plot_surface(X, Y, Zpos, color="blue")
16 ax.plot_surface(X, Y, Zneg, color="blue")
17 plt.show()
```

La surface obtenue est un double cône (voir ci-dessous)

4)

```
1 def x(t):
2     return np.cos(t)
3 def y(t):
4     return np.sin(t)
5 T = np.arange(0, 2*np.pi, 0.01)
6 X = x(T)
7 Y = y(T)
8 plt.axis('equal') # Pour que le cercle ne soit pas déformé
9 plt.plot(X, Y)
10 plt.grid()
11 plt.show()
```

Les deux figures obtenues sont :

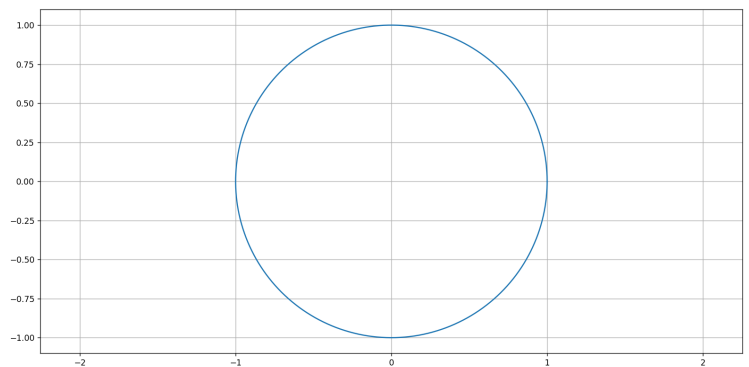
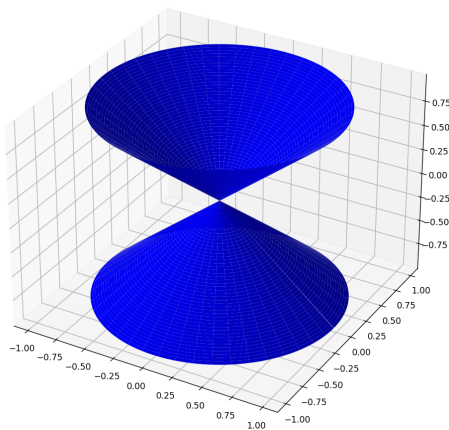


FIGURE 8 – Tracés des surfaces C (à gauche) et D (à droite)

Exercice 28 Soit $(F_n) \in \mathbb{N}^{\mathbb{N}}$ telle que $F_0 = 0$, $F_1 = 1$ puis $F_{n+2} = F_{n+1} + F_n$. Soit $M \in S_n(\mathbb{R})$.

Pour $n \geq 2$, on définit $A_n = (F_{i+j-2})_{1 \leq i, j \leq n} \in M_n(\mathbb{R})$.

b) Écrire une fonction python qui prend n en argument et renvoie A_n sous forme d'un tableau numpy.

c) Écrire une fonction python qui détermine les valeurs propres de A_n .

Corrigé 28 b)

```
1 def a(n):
2     # On genere les n premiers termes de F
3     fibonacci = np.zeros(2*n, dtype=int)
4     fibonacci[0] = 0
5     fibonacci[1] = 1
6     for i in range(2, 2*n):
7         fibonacci[i] = fibonacci[i-1] + fibonacci[i-2]
8     # On genere la matrice A
9     res = np.zeros((n,n), dtype=int)
10    for i in range(n):
11        for j in range(n):
12            res[i,j] = fibonacci[i+j]
13    return res
```

c)

```
1 # Il existe une fonction, appelee dans le document Matrice, qui le fait:
2 alg.eigvals(a(n))
3
4 #Sinon, via les racines du polynome caracteristique:
5 np.roots(np.poly(a(n)))
```

Exercice 29 Soit $A \in M_n(\mathbb{R})$.

c) Utilisation de python

1. Ecrire une fonction `randO(n)` qui génère une matrice aléatoire A et renvoie la matrice orthogonale O de la question précédente (telle que $A = OT$ avec T triangulaire supérieure).
2. Ecrire une fonction `N1`, de la variable matricielle A , qui renvoie $N_1(A)$. On pourra utiliser les fonction `numpy.sum` et `numpy.abs`.
3. Ecrire une fonction `test(n)` qui, sur 1000 tests, renvoie le minimum et le maximum des valeurs de N_1 , pour des matrices orthogonales aléatoires.

Corrigé 29

1. En utilisant la fonction donnée dans l'énoncé, on a

```
1 def randO(n):
2     a = np.zeros((n,n),dtype=int)
3     for i in range(n):
4         for j in range(n):
5             a[i,j] = rd.randint(-100,100)
6     return np.linalg.qr(a)[0]
```

2. En utilisant les deux fonctions suggérées, on a

```
1 def n1(a):
2     return np.sum(np.abs(a))
```

3. En utilisant les fonctions écrites préalablement, on a

```
1 def test(n):
2     minimum = n1(randO(n))
3     maximum = minimum
4     for _ in range(999):
5         valeur = n1(randO(n))
6         if valeur > maximum:
7             maximum = valeur
8         if valeur < minimum:
9             minimum = valeur
10    return minimum,maximum
```

Exercice 30 NON FAIT

Exercice 31 Soit S la sphère unité de R_n . Pour $M \in S_n(\mathbb{R})$, on note $\lambda_1(M), \dots, \lambda_n(M)$ ses valeurs propres triées dans l'ordre croissant. Pour un sous-espace vectoriel F non nul de E , on note $R_M(F) = \max_{X \in F \cap S} X^T M X$.

1. Écrire une fonction qui, pour $M \in S_n(\mathbb{R})$, renvoie la liste des valeurs propres triées par ordre croissant : $[\lambda_1(M), \dots, \lambda_n(M)]$.
2. Écrire une fonction qui, pour $(A, B) \in S_n(\mathbb{R})^2$ avec $A \neq B$, renvoie la liste des $\frac{|\lambda_k(A) - \lambda_k(B)|}{\max\{-\lambda_1(A-B), \lambda_n(A-B)\}}$ pour $k \in \llbracket 1, n \rrbracket$.
3. Tester cette fonction pour des matrices symétriques aléatoirement choisies.

Corrigé 31

1. La fonction `alg.eigvals` renvoie les valeurs propres d'une matrice

```
1 def valeurs_propres(m):
2     return np.sort(alg.eigvals(m))
```

2. En utilisant la fonction précédente

```
1 def liste(a,b):
2     valeurs_propres_a = valeurs_propres(a)
3     valeurs_propres_b = valeurs_propres(b)
4     valeurs_propres_a_b = valeurs_propres(a-b)
5     return np.abs(valeurs_propres_a - valeurs_propres_b) / max(-valeurs_propres_a_b[0],
        valeurs_propres_a_b[-1])
```

3. Il suffit de générer des matrices symétriques aléatoirement

```
1 def test(n):
2     a = np.zeros((n,n),dtype=int)
3     b = np.zeros((n,n),dtype=int)
4     for i in range(n):
5         for j in range(i,n):
6             a[i,j] = rd.randint(-100,100)
7             b[i,j] = rd.randint(-100,100)
8             a[j,i] = rd.randint(-100,100)
9             b[j,i] = rd.randint(-100,100)
10    return liste(a,b)
```

7 Intégration

Exercice 40 Soit $U : \mathbb{N}^2 \rightarrow \mathbb{N}$ telle que :

$$U(0, 0) = 1.$$

$$\forall n \in \mathbb{N}^*, U(0, n) = U(n, 0) = 0.$$

$$\forall n, m \in \mathbb{N}^*, U(n, m) = U(n-1, m-1) + m U(n-1, m).$$

1. a) Écrire une fonction Python qui renvoie $U(n, m)$. Calculer $U(6, 4)$.

c) Soit pour $n \in \mathbb{N}^*$, $P_n = X(X-1) \dots (X-n+1)$ et $P_0 = 1$. Calculer $\sum_{k=0}^n U(n, k) P_k$ pour $n \in \llbracket 0, 10 \rrbracket$.

Corrigé 40 1. a)

```
1 def u(n,m):
2     if (n,m)==(0,0):
3         return 1
4     if n==0 or m==0:
5         return 0
6     return u(n-1,m-1)+m*u(n-1,m)
```

c)

```
1 def somme(n):
2     cmpt = Polynomial([0])
3     polynome = Polynomial([1])
4     for k in range(n + 1):
5         cmpt += u(n, k) * polynome
6         polynome = polynome * Polynomial([-k,1])
7     return cmpt
8
9 for n in range(1,11):
10     print("n =",n,":",somme(n))
```

et on peut conjecturer que $\text{somme}(i) = X^i$ pour tout $i \in \mathbb{N}$.

8 Equations différentielles

Exercice 53 On définit sur $M_n(\mathbb{C})$ la norme : $\forall M = [m_{i,j}]_{1 \leq i,j \leq n} \in M_n(\mathbb{C}), \|M\| = \max_{1 \leq i \leq n} \left(\sum_{j=1}^n |m_{i,j}| \right)$.

1. En PYTHON, écrire une fonction représentant cette norme.

2. Soit $\phi : X \mapsto AX + XB$ définie sur $M_n(\mathbb{C})$, avec $A, B \in M_n(\mathbb{C})$. Écrire une fonction en PYTHON, qui prend en argument A et B et renvoie la matrice de ϕ dans la base canonique de $M_n(\mathbb{C})$.

Corrigé 53 1.

```
1 def norme(m):
2     return np.amax(np.sum(np.abs(m)[i] for i in range(m.shape[0])))
```

2.

```
1 def phi(x,a,b):
2     return np.dot(a,x) + np.dot(x,b)
3
4 def matrice_phi(a,b): # La matrice dans la base canonique est la transpose de la matrice dont toutes les
5     lignes sont phi(E11), phi(E12), ...
6     matrice = []
7     n = a.shape[0]
8     for i in range(n**2):
9         x = np.array([0]*n**2) # On cree une matrice elementaire sous la forme d'un tableau que l'on
10            redimensionne en matrice n*n
11         x[i]=1
12         matrice.append(phi(x.reshape((n,n)),a,b).reshape(n**2)) # On redimensionne la matrice obtenue en
13            tableau de taille n**2
14     return np.transpose(np.array(matrice))
```

Exercice 54 On pose pour tout $n \in \mathbb{N}$ et pour tout $x \in \mathbb{R}$: $J_n(x) = \int_0^{2\pi} \cos(x \sin(t) - nt) dt$.

On appelle zéros de J_n tout $x_0 \in \mathbb{R}$ tel que $J_n(x_0) = 0$.

1) Python : Écrire une fonction pour définir J_n et afficher J_n pour $x \in [0, 20]$ et $n \in \llbracket 0, 3 \rrbracket$.

On admet que J_n s'annule sur tout segment de longueur 1.

2) Python : définir une fonction `zeros(n,a)` qui renvoie les listes des ordonnées de zéros de J_n sur $[0, a]$.

Corrigé 54 1)

```

1 def j(x,n):
2     def f(t):
3         return np.cos(x * np.sin(t) - n*t)
4     return integr.quad(f,0,2*np.pi)[0]
5
6 j = np.vectorize(j)
7 X = np.arange(0,20,0.01)
8 Y0 = j(X,0)
9 Y1 = j(X,1)
10 Y2 = j(X,2)
11 Y3 = j(X,3)
12 plt.plot(X,Y0,label="J0",color="blue")
13 plt.plot(X,Y1,label="J1",color="red")
14 plt.plot(X,Y2,label="J2",color="green")
15 plt.plot(X,Y3,label="J3",color="orange")
16 plt.legend()
17 plt.grid()
18 plt.show()

```

et on obtient le graphique suivant :

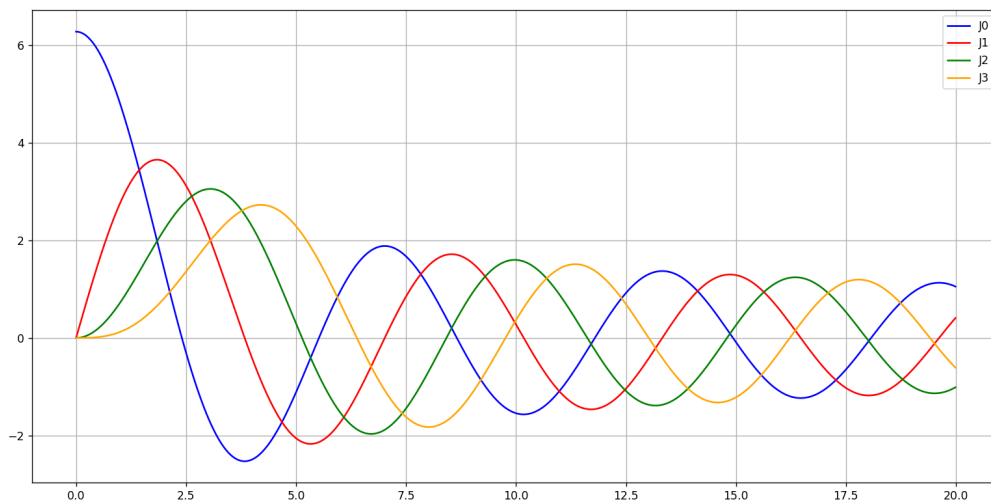


FIGURE 9 – Tracé de $J_n(x)$ pour différentes valeurs de n

2)

```

1 def zeros(n,a):
2     liste_zeros = []
3     for x in np.arange(0.01,a,0.01):
4         if j(x,n) * j(x-0.01,n) <= 0: # Si la fonction a change de signe, alors elle s'est annulée d'après
5             le TVI
6             liste_zeros.append(x)
7     return np.array(liste_zeros)

```

Exercice 55 On note, pour tout $n \in \mathbb{N}^*$, $a_n = \frac{n^{n-1}}{n!}$. On note $W(x)$ la somme de la série $\sum a_n x^n$.

2) Tracer sur Python les 300 premières sommes partielles.

4) On note, pour tout $p \in \mathbb{N}^*$, $S_p = \sum_{k=0}^p \binom{p}{k} (-1)^k k^{p-1}$. Calculer avec Python les 10 premiers termes de la suite $(S_p)_{p \in \mathbb{N}^*}$.

Corrigé 55 2)

```

1 def a(n):
2     return n**(n-1) / np.math.factorial(n)
3
4 X = np.arange(-0.4, 0.4, 0.01)
5 somme_partielles = np.zeros((300, len(X)))
6 for n in range(1, 301):
7     somme_partielles[n - 1] = somme_partielles[n - 2] + a(n) * (X ** n) if n > 1 else a(n) * X
8
9 for n in range(10, 301, 30):
10    plt.plot(X, somme_partielles[n - 1], label=f'N = {n}')
11 plt.legend()
12 plt.grid()
13 plt.show()

```

et on obtient le graphique suivant :

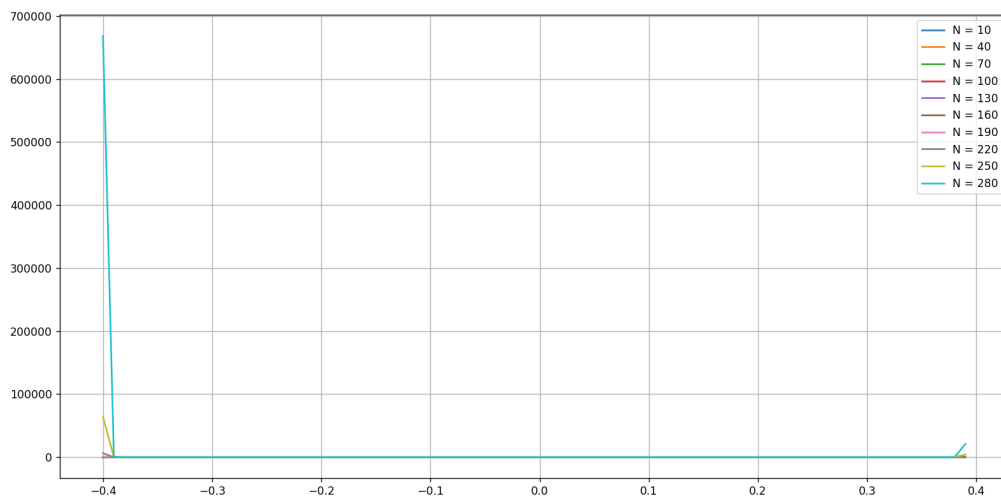


FIGURE 10 – Tracé des 300 premières sommes partielles

4)

```

1 def binom(k,n):
2     return np.math.factorial(n) // (np.math.factorial(k) * np.math.factorial(n-k))
3
4 def s(p):
5     return np.sum(np.array([binom(k, p) * (-1)**k * (k**(p-1) if k != 0 else 1) for k in range(p + 1)]))
6
7 for i in range(1,11):
8     print("S",i,"=",s(i))

```

9 Intégrales à paramètre

Exercice 56 Tracer la courbe représentative de la fonction f définie par $f(x) = \int_0^{2\pi} \cos(x \sin(t)) dt$.

```
1 def f(x):
2     def g(t):
3         return np.cos(x * np.sin(t))
4     return integr.quad(g,0,np.pi/2)[0]
5
6 f = np.vectorize(f)
7 X = np.arange(-150,150,0.1)
8 Y = f(X)
9 plt.plot(X,Y)
10 plt.title("Trace de f")
11 plt.show()
```

et on obtient le graphique suivant :

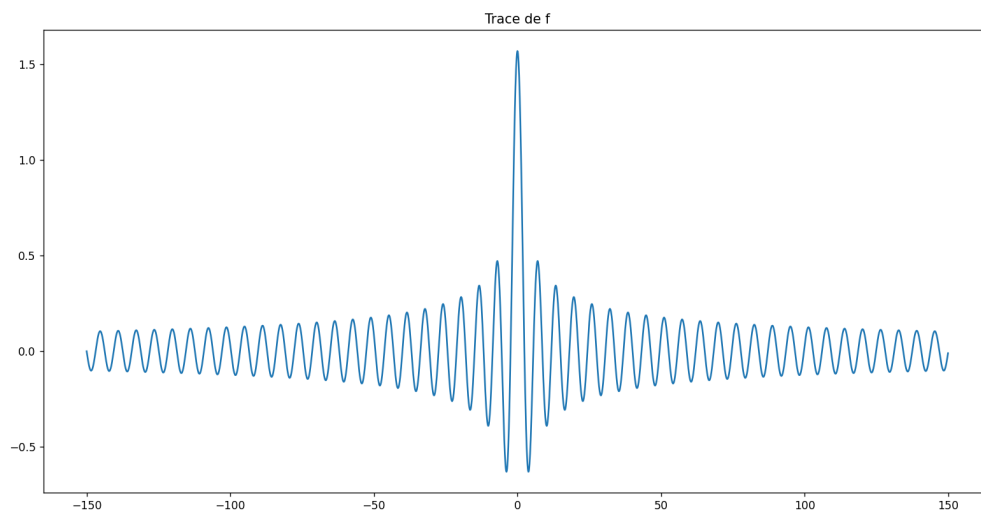


FIGURE 11 – Tracé de la fonction f

Exercice 57 1. a) Écrire un programme Python qui pour un entier naturel non nul n associe le nombre de diviseurs positifs $d(n)$.

b) On pose : $\forall x \in]-1, 1[$, $\forall n \in \mathbb{N}^*$, $u_n(x) = \frac{x^n}{1-x^n}$ et $f(x) = \sum_{n=1}^{+\infty} u_n(x)$. Écrire un programme Python qui donne une approximation de $f(x)$ et comparer à $\sum_{n=1}^{+\infty} d(n)x^n$. Conjecture?

Corrigé 57 1. a)

```
1 def d(n):
2     cmpt = 0
3     for d in range(1,n+1):
4         if n%d==0:
5             cmpt += 1
6     return cmpt
```

b)

```
1 def u(n,x):
2     return x**n/(1-x**n)
3
4 def f(x,N):
5     return np.sum([u(n,x) for n in range(1,N)])
6
7 def g(x,N):
8     return np.sum([d(n)*x**n for n in range(1,N)])
9
10 f = np.vectorize(f)
11 g = np.vectorize(g)
12 N = 500
13 X = np.arange(-0.99,0.99,0.01)
14 Y = f(X,N)
15 Z = g(X,N)
16 plt.plot(X,Y,label="f(x)",color="blue")
17 plt.plot(X,Z,label="g(x)",color="orange")
18 plt.legend()
19 plt.show()
```

et on obtient le graphique suivant :

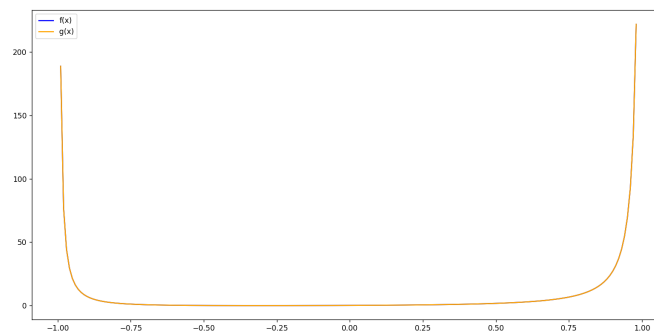


FIGURE 12 – Tracé de la fonction f et de la fonction g

Les deux graphes se superposent : on peut conjecturer que $f = g$ pour tout $x \in]-1 ; 1[$.

Exercice 58 Soit $\beta > 0$. On pose, pour $x \in \mathbb{R}$: $C(x) = \int_0^{+\infty} \frac{\cos(xt)}{1+t^2} dt$ et $I_\beta(x) = \int_0^{+\infty} \frac{\cos(xt)e^{-\beta t}}{1+t^2} dt$.

a) Définir une fonction qui calcule C.

b) Représenter $x \mapsto C(x)e^x$, conjecturer une expression de C sur $\mathbb{R}$.

Corrigé 58 a)

```
1 def c(x):
2     def f(t):
3         return np.cos(x*t) / (1 + t**2)
4     return integr.quad(f,0,np.inf)[0]
```

b)

```
1 def g(x):
2     return c(x) * np.exp(x)
3
4 g = np.vectorize(g)
5
6 X = np.arange(-10,10,0.01)
7 Y = g(X)
8 plt.plot(X,Y)
9 plt.show()
```

et on obtient le graphique suivant :

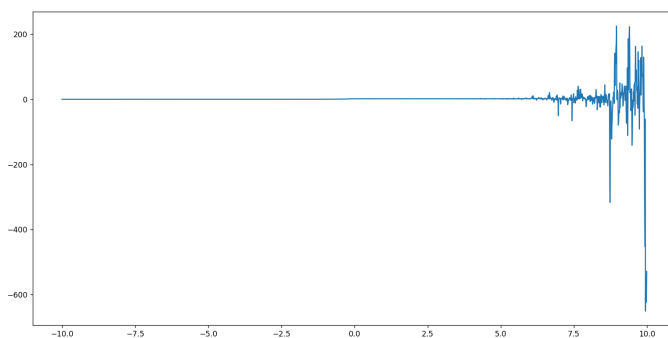


FIGURE 13 – Tracé de la fonction $C(x)e^x$

On peut conjecturer que C est la fonction nulle sur $\mathbb{R}$.

10 Structures algébriques, polynômes et arithmétique

Exercice 59 On appelle "faux premier" tout entier $n \geq 2$ non premier vérifiant la propriété suivante :

$$(C) : \forall a \in \mathbb{Z}, a \wedge n = 1 \Rightarrow a^{n-1} \equiv 1[n]$$

Notons que la propriété (C) est équivalente à la propriété suivante :

$$(C') : \forall a \in \llbracket 2, n \rrbracket, a \wedge n = 1 \Rightarrow a^{n-1} \equiv 1[n].$$

L'objectif de l'exercice est de démontrer que les faux premiers sont les entiers $n \geq 2$ vérifiant les trois conditions suivantes :

- n n'est pas premier.
 - n n'a aucun facteur premier multiple.
 - Pour tout facteur premier p de n , on a que $p - 1$ divise $n - 1$.
1. Montrer que tout faux premier est impair. Indication : on pourra considérer l'entier -1 .
 2. Coder en python une fonction `pgcd(a, b)` qui renvoie le pgcd des entiers a et b .
 3. Coder en python une fonction `crible(n)` qui une liste L de Booléens, de taille n , et telle que l'élément d'indice k de L soit `True` si k n'est pas premier, et `False` sinon. Afficher tous les faux premiers inférieurs à 10000

Corrigé 59 2.

```
1 def pgcd(a,b):
2     if b==0:
3         return a
4     return pgcd(b,a%b)
```

3.

```
1 def crible(n):
2     res = np.full(n+1, False) # Permet d'obtenir un array rempli de False
3     # Initialisation pour 0 et 1
4     res[0] = True
5     if n==0:
6         return res
7     res[1] = True
8     for i in range(2,n+1):
9         for j in range(i*2,n+1,i): # On commence a 2i et on augmente de i a chaque iteration (donc 3i, 4i
10             ...
11             res[j] = True
12     return res
```

```
1 def facteurs_preiers(a): # Renvoie un array vide si a a un facteur premier multiple
2     premiers = crible(a)
3     facteurs = []
4     for i in range(a):
5         if not premiers[i] and a%i == 0: # Si i est premier et divise a
6             if a%(i*i) == 0: # Si i est un facteur premier multiple
7                 return np.array([])
8             facteurs.append(i)
9     return np.array(facteurs)
10
11 faux_preiers = []
12 for i in range(3,10000,2): # Les faux premiers sont impairs d'apres la question 1
13     facteurs = facteurs_preiers(i)
14     taille = np.shape(facteurs)[0]
15     est_faux_preier = True
16     if taille==0: # Si la liste est vide, c'est qu'il est premier ou qu'il a un facteur premier multiple
17         donc on arrete d'etudier ce nombre
18         est_faux_preier = False
19     for j in range(taille):
20         if (i-1)%(facteurs[j]-1) != 0: # Condition 3
21             est_faux_preier = False
22     if est_faux_preier:
23         faux_preiers.append(i)
24 faux_preiers = np.array(faux_preiers)
```

Exercice 60 Une permutation σ sera représentée par une liste Python : $[\sigma(1), \dots, \sigma(n)]$

On appelle Dérangement toute permutation $\sigma \in \mathfrak{S}_n$ telle que son support soit $\llbracket 1, n \rrbracket$ (c'est-à-dire sans point fixe). On notera par la suite d_k le nombre de dérangements de $\mathfrak{S}_k$

Partie Python :

1. Programmer une fonction *est_derangement* prenant en entrée une permutation σ déterminant si σ est un dérangement.
2. Programmer une fonction *permutations* prenant en entrée un entier n renvoyant toute les permutations de $\mathfrak{S}_n$ (On pourra procéder de manière récursive).
3. Programmer une fonction *derangements* prenant en entrée un entier n renvoyant le nombre de dérangements de $\mathfrak{S}_n$.

Corrigé 60 1.

```
1 def est_derangement(sigma):
2     n = sigma.shape[0]
3     # On verifie s'il contient une fois chaque valeur de 1 à n
4     verif = np.full(n, False)
5     for i in range(n):
6         # On verifie en plus s'il ne possède pas de point fixe
7         if sigma[i]-1 == i:
8             return False
9         verif[sigma[i]-1] = True
10    if not np.array_equal(verif, np.full(n, True)):
11        return False
12    return True
```

2.

```
1 toutes_permutations = []
2 def aux(n, perm, used):
3     if len(perm) == n:
4         toutes_permutations.append(np.array(perm)) # On transforme en array, ce qui retire les effets de
5         bord
6     else:
7         for i in range(1, n + 1):
8             if not used[i]:
9                 used[i] = True
10                perm.append(i)
11                aux(n, perm, used)
12                perm.pop()
13                used[i] = False
14 def permutations(n):
15     global toutes_permutations
16     toutes_permutations = []
17     used = np.full(n+1, False)
18     aux(n, [], used)
19     return toutes_permutations
```

3.

```
1 def derangements(n):
2     cmpt = 0
3     global toutes_permutations
4     permutations(n)
5     for sigma in toutes_permutations:
6         if est_derangement(sigma):
7             cmpt += 1
8     return cmpt
```

Exercice 61 Soit $n \in \mathbb{N}$. On note D_n l'ensemble des diviseurs positifs de n . Soit $\sigma(n) = \sum_{d \in D_n} d$. On note Z l'ensemble des $n \in \mathbb{N}$ tels que $D_n = A \cup B$ avec A et B disjoints tels que $\sum_{d \in A} d = \sum_{d \in B} d$.

1. Écrire une fonction Python qui renvoie la liste des diviseurs positif d'un entier n .
2. Écrire une fonction Python qui renvoie la liste des entiers n entre 1 et 100 appartenant à Z .

Commentaires divers : 2)a) Pour le code, une fonction renvoyant la liste des sous-listes d'une liste était donnée.

Corrigé 61 1.

```
1 def diviseurs(n):
2     res = []
3     for i in range(1,n+1):
4         if n%i == 0:
5             res.append(i)
6     return np.array(res)
```

2.

```
1 # D'après le commentaire, une fonction semblable etait donnee
2 def aux(l, sl, liste, index):
3     if index == len(l):
4         liste.append(np.array(sl))
5     else:
6         aux(l, sl, liste, index + 1)
7         sl.append(l[index])
8         aux(l, sl, liste, index + 1)
9         sl.pop()
10
11 def liste_sous_listes(l):
12     liste = []
13     aux(l, [], liste, 0)
14     return liste
15
16 def appartient_Z():
17     res = []
18     for n in range(1,101):
19         div = diviseurs(n)
20         sous_listes = liste_sous_listes(div)
21         # On verifie pour chaque sous-liste si elle correspond un partitionnement qui convient
22         for sl in sous_listes:
23             cmpt_a = 0
24             cmpt_b = 0
25             index = 0
26             for i in range(len(div)):
27                 if index < len(sl) and div[i] == sl[index]:
28                     cmpt_a += div[i]
29                     index += 1
30             else:
31                 cmpt_b += div[i]
32             if cmpt_a == cmpt_b:
33                 res.append(n)
34                 break # Cela ne sert a rien de continuer a explorer les sous listes si on en a trouve une
                       # qui convient
35     return np.array(res)
```

Exercice 62 Pour tout entier $n \in \mathbb{N}^*$, on note $\mathbb{U}_n$ l'ensemble des racines n -ièmes de l'unité et Π_n l'ensemble des racines primitives de l'unité (c'est-à-dire des générateurs de $\mathbb{U}_n$).

Après avoir fixé $n \in \mathbb{N}^*$, on notera par la suite $\omega = e^{\frac{2i\pi}{n}}$. Pour p un nombre premier, on notera $\Phi_p = \prod_{\xi \in \Pi_n} (X - \xi)$

b) Sur Python, afficher les Φ_p pour p premier de 2 à 13.

Indication(s) fournie(s) par l'examineur pendant l'épreuve : L'examineur a clarifié pour moi le sens de la question 1. b), il était attendu qu'on affiche non pas la courbe représentative des polynômes, mais leurs formules avec les coefficients.

Corrigé 61 b)

```
1 def phi(p):
2     res = Polynomial([-1,1]) # X - xi pour xi = 1
3     for k in range(1,p):
4         res = res * Polynomial([-np.exp(2j*k*np.pi/p),1])
5     return res
6
7 print("p = 2:",phi(2))
8 print("p = 3:",phi(3))
9 print("p = 5:",phi(5))
10 print("p = 7:",phi(7))
11 print("p = 11:",phi(11))
12 print("p = 13:",phi(13))
```

Exercice 63 Soit n un entier naturel et $V = \mathbb{C}_n[X]$.

Pour $g = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, on note pour tout polynôme P , $P_g = (cX + a)^n P\left(\frac{-dX+b}{-cX+a}\right)$.

Soit ϕ_g l'application qui à P associe P_g .

b) Écrire une fonction `action(g, P)` qui retourne P_g (on pourra utiliser le module `Polynomial`). On donnait un exemple et le résultat attendu pour vérifier le fonctionnement.

c) Écrire une fonction `mat(g)` qui renvoie la matrice de ϕ_g dans la base canonique. La lancer sur S , U et T , calculer les valeurs propres à chaque fois (matrices simples données au début, avec que des 0, 1 et -1)

Corrigé 63 b)

```
1 n=4
2 def action(g,P):
3     polynome_puissance = Polynomial([g[0,0],-g[1,0]])**n # Pour elever un polynome a la puissance n, il faut
        utiliser **n
4     polynome_compose = Polynomial([g[0,1],-g[1,1]]) // Polynomial([g[0,0],-g[1,0]]) # Attention: pour
        diviser deux polynomes, il faut utiliser //
5     return polynome_puissance * P(polynome_compose)
```

c)

```
1 def mat(g):
2     # La transposee de la matrice voulue est la matrice dont toutes les lignes sont action(g,1), action(g,X)
        , ..., action(g,X^n)
3     matrice = []
4     for i in range(n+1):
5         matrice.append(action(g,Polynomial([0]*i+[1])).coef)
6     return np.transpose(np.array(matrice))
7
8 alg.eigvals(mat(g))
```

Exercice 64 Soit $p > 2$ un nombre premier : on sait alors que $(\mathbb{Z}/p\mathbb{Z})$ est un corps. Pour $k \in \mathbb{Z}$, on note $\bar{k}$ son représentant modulo p . On pose alors $(\mathbb{Z}/p\mathbb{Z})^* = (\mathbb{Z}/p\mathbb{Z}) \setminus \{\bar{0}\}$.

On admet que pour tout polynôme P de degré d dans un corps, P admet au plus d racines dans ce corps.

On note enfin $C = \{x^2 \mid x \in (\mathbb{Z}/p\mathbb{Z})^*\}$.

1. a) Écrire une fonction Python notée *ensemble_c* qui prend comme paramètre un nombre premier p , et qui renvoie une liste constituée de tous les $k \in \llbracket 1, p \rrbracket$ vérifiant $k = x^2$, où $x \in (\mathbb{Z}/p\mathbb{Z})^*$.

b) Pour tous les nombres premiers entre 3 et 100, calculer $\text{card}(C)$ avec Python. Que peut-on conjecturer?

Corrigé 64 1. a)

```
1 def ensemble_c(p):
2     res = []
3     deja_present = np.full(p, False) # Permet d'obtenir un array rempli de False
4     for x in range(1, p):
5         if not deja_present[x*x%p]:
6             res.append(x*x%p)
7             deja_present[x*x%p] = True
8     return np.array(res)
```

b)

```
1 def crible(n):
2     res = np.full(n+1, True) # Permet d'obtenir un array rempli de True
3     # Initialisation pour 0 et 1
4     res[0] = False
5     if n==0:
6         return res
7     res[1] = False
8     for i in range(2, n+1):
9         for j in range(i*2, n+1, i): # On commence a 2i et on augmente de i a chaque iteration (donc 3i, 4i ...)
10            res[j] = False
11     return res
12
13 est_premier = crible(100)
14 for i in range(100):
15     if est_premier[i]:
16         print("p =", i, "et card(c) =", ensemble_c(i).shape[0])
```

On peut conjecturer que le cardinal de C vaut la moitié de p (arrondi à l'inférieur)

Exercice 65 Soit G un sous groupe de $O_n(\mathbb{R})$, on définit la trace de G par $\text{tr}(G) = \{\text{tr}(A), A \in G\}$. On dit que G est d'exposant fini s'il existe $m \in \mathbb{N}^*$ tel que pour tout $A \in G$, $A^m = I_n$. On note $R(\theta)$ la matrice de rotation d'angle θ .

1. a) Écrire une fonction sur Python qui renvoie la matrice de rotation d'angle θ .

b) Soit deux matrices A et B , écrire une fonction sur Python qui calcule le groupe engendré par les matrices A et B .

Pour que la complexité ne soit pas très élevée : si le groupe engendré possède plus de 100 éléments ou est infini, l'algorithme ne devra renvoyer que les 100 premières matrices calculées. pour parcourir la liste L de matrices construites, on utilisera `for C in L[:]` plutôt que `for C in L`. (également une autre consigne dont je ne me rappelle plus)

c) Vérifier que le groupe engendré par $R_{\frac{\pi}{3}}$ et $R_{\frac{\pi}{5}}$ est bien fini et possède 30 éléments.

Corrigé 65 1. a)

```
1 def rotation(theta):
2     return np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
```

b)

```
1 def sous_groupe_engendre(a,b):
2     matrices_simples = [a,b]
3     sous_groupe = [np.eye(a.shape[0])]
4     if alg.det(a) != 0:
5         matrices_simples.append(alg.inv(a)) # On ajoute l'inverse de a si elle est inversible
6     if alg.det(b) != 0:
7         matrices_simples.append(alg.inv(b)) # On ajoute l'inverse de b si elle est inversible
8     cmpt = 0
9     nb_matrices_ajoutees = 1 # Representant le nombre de matrices ajoutes lors de la derniere iteration
10    while nb_matrices_ajoutees > 0 and len(sous_groupe)<100:
11        for i in range(nb_matrices_ajoutees): # On ajoute les matrices simples a toutes les matrices
12            # generees a la derniere iteration
13            for m in matrices_simples:
14                matrice = np.dot(sous_groupe[-1-i],m)
15                matrice_est_nouvelle = True
16                for n in sous_groupe[:]:
17                    if np.allclose(n,matrice, rtol=1e-10, atol=1e-10): #np.array_equal() ne fonctionne pas
18                        # ici a cause de la manipulation de flottants
19                        matrice_est_nouvelle = False
20                        break
21                if matrice_est_nouvelle:
22                    sous_groupe.append(matrice)
23                    cmpt += 1
24            nb_matrices_ajoutees = cmpt
25    cmpt = 0
26    return np.array(sous_groupe[:100])
```

c)

```
1 print(sous_groupe_engendre(rotation(np.pi/3),rotation(np.pi/5)).shape[0])
```

Exercice 66 Soit $n \in \mathbb{N}^*$. On pose $\sigma(n) = \sum_{d|n} d$, et $\tau(n) = \text{Card}\{d \in \mathbb{N} / d|n\}$.

1. Coder σ et τ en Python.
2. Numériquement, donner les $k \in \llbracket 1, 1000 \rrbracket$ tels que $\sigma \circ \tau(n) = n$, puis ceux tels que $\tau \circ \sigma(n) = n$.

Corrigé 66 1.

```
1 def sigma(n):
2     cmpt = 0
3     for d in range(1,n+1):
4         if n%d==0:
5             cmpt += d
6     return cmpt
7
8 def tau(n):
9     cmpt = 0
10    for d in range(1,n+1):
11        if n%d==0:
12            cmpt += 1
13    return cmpt
```

2.

```
1 sigma_tau = []
2 tau_sigma = []
3 for k in range(1,1001):
4     if sigma(tau(k))==k:
5         sigma_tau.append(k)
6     if tau(sigma(k))==k:
7         tau_sigma.append(k)
8 sigma_tau = np.array(sigma_tau)
9 tau_sigma = np.array(tau_sigma)
```


Exercice 67 1. a) Coder $\binom{n}{k}$.

b) On note $P(n)$ l'ensemble des nombres premiers inférieurs ou égaux à n et P_n le produit des éléments de $P(n)$.

Coder sur Python une fonction qui renvoie P_n , puis calculer $P_{\binom{4}{2}}$ et $P_{\binom{6}{3}}$.

Corrigé 67 1. a)

```
1 def binom(k,n):
2     return np.math.factorial(n) // (np.math.factorial(k) * np.math.factorial(n-k))
```

b)

```
1 def p(n):
2     cmpt = 1
3     res = np.full(n+1, True) # Permet d'obtenir un array rempli de True
4     for i in range(2,n+1):
5         for j in range(i*2,n+1,i): # On commence a 2i et on augmente de i a chaque iteration (donc 3i, 4i
6             ... )
7             res[j] = False
8         # Si i est premier, on multiplie
9         if res[i]:
10             cmpt *= i
11     return cmpt
12 print(p(binom(2,4)))
13 print(p(binom(3,6)))
```

Exercice 68 On fixe A un entier supérieur ou égal à 3. On note $a \text{ div } b$ et $a \text{ mod } b$ respectivement le quotient et le reste de la division euclidienne de a par b .

On définit la suite $(F_n)_{n \in \mathbb{N}}$ la suite telle que $F_0 = 0$, $F_1 = 1$ et $\forall n \in \mathbb{N}$, $F_{n+2} = F_{n+1} + F_n$.

On note : $\forall n \in \mathbb{N}$, $u_n = [A^{n^2+n} \bmod (A^{2n} - A^n - 1)] \bmod A^n$.

Utiliser Python pour calculer les vingt premiers termes de $(F_n)_{n \in \mathbb{N}}$ et de $(u_n)_{n \in \mathbb{N}}$. Que peut-on conjecturer?

```

1 def f(n):
2     res = np.zeros(n, dtype=int)
3     if n==0:
4         return res
5     res[1]=1
6     for i in range(2,n):
7         res[i] = res[i-1] + res[i-2]
8     return res
9
10 A = 3 # Exemple
11 def u(n):
12     res = np.zeros(n, dtype=int)
13     for i in range(n):
14         res[i] = (A**(i**2+i) % (A**(2*i) - A**i - 1)) % A**i
15     return res

```

On peut conjecturer que, peu importe la valeur de A , $f(n) = u(n)$

Exercice 69 1) On définit la famille $(F_n)_{n \in \mathbb{Z}}$ avec : $F_0 = 0, F_1 = 1$ et la relation : $\forall n \in \mathbb{Z}, F_{n+1} = F_n + F_{n-1}$. Coder une fonction renvoyant F_n pour n dans $\mathbb{Z}$. Afficher F_n pour $-10 \leq n \leq 10$.

3) Coder une fonction donnant le pgcd de deux entiers via l'algorithme d'Euclide, et le nombre de divisions euclidiennes effectuées. La tester avec des couples (F_{n+2}, F_{n+1}) pour $n > 0$. Commenter.

Corrigé 69 1)

```
1 def f(n):
2     val0 = 0
3     val1 = 1
4     for i in range(2,n+1):
5         temp = val1
6         val1 = val0 + temp
7         val0 = temp
8     for i in range(-n+1): # Il faut faire une iteration de plus pour que le r sultat soit dans val1
9         temp = val0
10        val0 = val1 - temp
11        val1 = temp
12    return val1
13
14 for i in range(-10,11):
15     print(f(i), end=" ; ")
```

3)

```
1 def aux(a,b,nb_div):
2     if b==0:
3         return a,nb_div
4     return aux(b,a%b,nb_div+1)
5
6 def pgcd(a,b):
7     res,cmpt = aux(a,b,0)
8     print("Nombre de divisions effectuees:",cmpt)
9     return a
10
11 for i in range(-10,11):
12     print(i,i+1, end=": ")
13     pgcd(f(i),f(i+1))
```

Le nombre de divisions effectuées augmente de 1 quand i augmente de 1.

On dirait que $\text{pgcd}(f(i),f(i+1))$ effectue i divisions (si $i > 0$)

11 Calcul différentiel

Exercice 70 On note, pour n dans $\mathbb{N}^*$, $\mathfrak{S}_n$ le groupe des permutations de $\{1, 2, \dots, n\}$. Pour M notée $(m_{i,j})_{1 \leq i,j \leq n}$ matrice carrée d'ordre n à coefficients réels, le permanent de M , noté $\text{Per}(M)$, est le nombre $\text{Per}(M) = \sum_{\sigma \in \mathfrak{S}_n} \prod_{i=1}^n m_{i,\sigma(i)}$.

On dit que M notée $(m_{i,j})_{1 \leq i,j \leq n}$ dans $M_n(\mathbb{R})$ est bistochastique si elle vérifie : $\forall i, j \in \{1, 2, \dots, n\}^2, m_{i,j} \geq 0, \forall i \in \{1, 2, \dots, n\}, \sum_{j=1}^n m_{i,j} = 1$ et $\forall j \in \{1, 2, \dots, n\}, \sum_{i=1}^n m_{j,i} = 1$. On pose $A = (\frac{1}{3})_{1 \leq i,j \leq 3}$.

1. a) Ecrire une fonction $\text{per}(M)$ qui calcule le permanent d'une matrice M carrée d'ordre 3. La vérifier sur la matrice A .

b) Soit $(a, b, c, d) \in \mathbb{R}^4$. Compléter si possible les coefficients de la matrice $\begin{pmatrix} a & b & . \\ c & d & . \\ . & . & . \end{pmatrix}$ pour qu'elle soit bistochastique, et

donner la condition sur (a, b, c, d) pour que ce soit possible.

Programmer une fonction Python qui renvoie une matrice bistochastique aléatoire d'ordre 3 (on rappelle que la fonction `numpy.random.random()` renvoie un flottant pseudo-aléatoire suivant une loi uniforme sur $[0, 1]$).

c) Écrire une fonction qui tire N matrices bistochastiques aléatoires d'ordre 3 et renvoie celle de permanent minimal (ainsi que son permanent). L'utiliser pour $N = 10000$. Que peut-on conjecturer?

Corrigé 70 1. a)

```
1 s_3 = np.array([np.array([1,2,3]),np.array([1,3,2]),np.array([2,1,3]),np.array([2,3,1]),np.array([3,1,2]),np
2 .array([3,2,1])]) # Liste des permutations de S_3
3
4 def perm(m):
5     cmpt = 0
6     for sigma in s_3:
7         temp = 1
8         for i in range(3):
9             temp *= m[i,sigma[i]-1]
10        cmpt += temp
11    return cmpt
12
13 a = np.full((3,3),1/3)
14 print(perm(a))
```

b) En omettant les détails, si on a $0.5 \leq a+b \leq 1, 0.5 \leq c+d \leq 1$ et de même pour les colonnes, on a un unique résultat pour les valeurs de chaque point (et le coin en bas à droite doit avoir la même valeur pour la colonne et pour la ligne).

```
1 def bistochastique():
2     matrice = np.zeros((3,3))
3     alea = (1 + rd.random()) / 2 # On genere un nombre aleatoire entre 0.5 et 1
4     matrice[0,0] = alea - (alea * rd.random()) # On genere un nombre aleatoire inferieur au nombre precedent
5     matrice[0,1] = alea - matrice[0,0] # On complete b pour que a+b = alea
6     matrice[0,2] = 1 - alea # Il ne nous reste qu'une unique facon de completer la ligne
7     alea = (1 + rd.random()) / 2 # On recommence mais pour la premiere colonne
8     matrice[1,0] = alea - matrice[0,0] # La valeur de a est deja fixee donc il ne reste plus que ca a faire
9     matrice[2,0] = 1 - alea
10    alea = (1 + rd.random()) / 2 # On complete la deuxieme ligne
11    matrice[1,1] = alea - matrice[1,0]
12    matrice[1,2] = 1 - alea
13    matrice[2,1] = 1 - matrice[1,1] - matrice[0,1]
14    matrice[2,2] = 1 - matrice[2,0] - matrice[2,1]
15    return matrice
```

c) On peut conjecturer que la matrice A est la matrice de permanent minimal

```
1 def permanent_minimum(N):
2     matrice_mini = bistochastique()
3     perm_mini = perm(matrice_mini)
4     for _ in range(N-1):
5         matrice = bistochastique()
6         if perm(matrice) < perm_mini:
7             perm_mini = perm(matrice)
8             matrice_mini = matrice
9     return matrice_mini, perm_mini
```